PROG5
Projet Documentation

Organisation du Projet ELF

Outil de Fusion de Fichiers Objets ELF32

Membres du Groupe

Ouassim Zouiten

Amir Majour

Mohamed Ilyes Amar Djarboa

Mouheidin Lakhdar Bougerra

Mohamed Ilyes Sahraoui

Wassim Firtah

Année Universitaire

2025/2026

Table des matières

1 Organisation des fichiers et arborescence du projet **2**
1.1 Racine du projet . . . 2
1.2 Répertoire project_folder/ . . . 2
1.2.1 Fichiers « étape » . . . 2
1.2.2 Programme principal (main.c) . . . 2
1.2.3 Modules de logique du projet . . . 3
1.2.4 Répertoire Tests/ . . . 4
1.3 Utilisation des exécutables . . . 4

2 Organisation générale du code **4**
2.1 Définitions ELF de base (ELF.h) . . . 4
2.2 Lecture ELF bas niveau (readELF.h) . . . 5
2.3 Rôle principal . . . 5
2.4 Fonctions clés . . . 5
2.5 Contexte ELF et chargement objet (ELFcntx.h, elf_object.h) . . . 5
2.5.1 ELFcntx . . . 5
2.5.2 elf_object_t . . . 6
2.6 Affichage ELF (Phase 1 – readelf simplifié) . . . 6
2.7 Fusion des sections (section_merge.h, section_merge_types.h) . . . 6
2.7.1 Objectif . . . 6
2.7.2 Principe . . . 6
2.7.3 Structures clés . . . 6
2.8 Fusion des symboles (symbol_merge.h, symbol_merge_types.h) . . . 6
2.8.1 Objectif . . . 6
2.8.2 Règles appliquées . . . 7
2.8.3 Résultat . . . 7
2.9 Fusion des relocations (realocation_merge.h) . . . 7
2.9.1 Objectif . . . 7
2.9.2 Traitements effectués . . . 7
2.10 Écriture du fichier ELF final (writeELF.h) . . . 7
2.10.1 Objectif . . . 7
2.10.2 Structure merged_elf_t . . . 7
2.10.3 Fonctions principales . . . 8

3 Fonctionnement global du projet **8**

4 Diagramme de Gantt - Progression **9**

I. Organisation des fichiers et arborescence du projet

Le projet est organisé en deux niveaux principaux :
— la racine du projet, contenant l'infrastructure de compilation et les fichiers utilitaires ;
— le répertoire `project_folder/`, qui contient l'intégralité du code source développé pour le projet.

I.1. Racine du projet

À la racine du projet, on trouve principalement des fichiers liés à l'infrastructure de compilation, à la documentation générale et aux scripts utilitaires.

Le fichier `README` à la racine explique :
— comment compiler l'ensemble du projet ;
— comment lancer chaque exécutable correspondant à une étape donnée.

I.2. Répertoire project_folder/

Le répertoire `project_folder/` contient l'ensemble du code source du projet. Les fichiers qu'il contient peuvent être regroupés en quatre catégories principales.

I.2.a) Fichiers « étape »

Les fichiers dont le nom suit le motif :

`etapeX_nom.c`

correspondent aux différentes étapes du projet (de l'étape 1 à l'étape 9).

Ces fichiers :
— ne contiennent pas de logique métier complexe
— servent uniquement de programmes d'illustration et de validation
— permettent d'exécuter une étape précise du projet de manière indépendante

Ils appellent les fonctions définies dans les modules du projet afin :
— d'afficher le résultat d'une étape donnée,
— ou de démontrer son fonctionnement isolément.

Ils peuvent être vus comme des instanciations successives du projet, montrant l'état du travail étape par étape.

I.2.b) Programme principal (main.c)

Le fichier `main.c` correspond au programme final du projet.
— Il ne contient pas de logique ELF interne
— Il agit comme un orchestrateur global
— Il enchaîne l'ensemble des étapes de lecture, fusion et écriture dans un seul exécutable

Ce programme représente donc la version finale et complète de l'outil de fusion ELF.

Comme les programmes d'étape, il peut être lancé :
— sans argument (affichage d'un message d'aide),
— ou avec `-help`.

I.2.c) Modules de logique du projet

Les autres fichiers `.c` et `.h` sont organisés par rôle fonctionnel, et contiennent la logique réelle du projet.

On y trouve notamment :

Définitions et parsing ELF

- `ELF.h`
- `readELF.h`, `readELF.c`
- `ELFcntx.h`, `ELFcntx.c`
- `elf_object.h`, `elf_object.c`

Affichage ELF

- `printELF.h`, `printELF.c`

Fusion des sections

- `section_merge.h`, `section_merge.c`
- `section_merge_types.h`
- `section_index.h`, `section_index.c`

Fusion des symboles

- `symbol_merge.h`, `symbol_merge.c`
- `symbol_merge_types.h`, `symbol_merge_types.c`

Fusion des relocations

- `realocation_merge.h`, `realocation_merge.c`

Écriture du fichier ELF

- `writeELF.h`, `writeELF.c`

Outils utilitaires et débogage

- `util.h`, `util.c`
- `debug.h`, `debug.c`

Ces modules sont réutilisés par tous les programmes d'étape ainsi que par `main`, ce qui garantit une séparation claire entre :

- la logique métier (fusion ELF),
- et les programmes de démonstration ou d'exécution.

I.2.d) Répertoire Tests/

Le répertoire `Tests/` contient l'ensemble des scripts de test automatisés développés au cours du projet.

Ces scripts ont été utilisés pour :

— valider le comportement de chaque étape individuellement,
— comparer les résultats avec les outils GNU (`arm-none-eabi-readelf`, `ld`),
— automatiser la vérification des sorties.

I.3. Utilisation des exécutables

Chaque exécutable correspondant à une étape :

— peut être lancé sans argument, auquel cas un message d'usage est affiché ;
— accepte également des options (selon l'étape concernée).

Le programme principal (`main`) se comporte de la même manière :

— affichage de l'aide sans argument,
— ou via l'option `-help`.

Les détails précis de compilation et d'exécution sont documentés dans le fichier `README` à la racine du projet.

II. Organisation générale du code

Le code est organisé autour de quatre grands blocs fonctionnels :

1. Lecture et parsing ELF
2. Affichage
3. Fusion des données ELF
4. Écriture du fichier ELF résultat

Chaque bloc repose sur des structures de données clairement définies dans les fichiers `.h`.

II.1. Définitions ELF de base (ELF.h)

Le fichier `ELF.h` constitue la base du projet. Il réimplémente les structures et constantes nécessaires du format ELF32 basé sur la documentation fournie.

Il définit notamment :

— les types ELF ABI-compatibles (`Elf32_Addr`, `Elf32_Word`, etc.)
— les structures principales :
  — `Elf32_Ehdr` (en-tête ELF)
  — `Elf32_Shdr` (en-tête de section)
  — `Elf32_Sym` (entrée de table des symboles)
  — `Elf32_Rel` / `Elf32_Rela` (relocations)
— les constantes :
  — types de sections (`SHT_PROGBITS`, `SHT_SYMTAB`, ...)
  — types et portées des symboles
  — types de relocations ARM (`R_ARM_ABS32`, `R_ARM_CALL`, `R_ARM_JUMP24`, ...)

Des assertions statiques garantissent la conformité ABI des structures (tailles exactes).

Ce fichier est utilisé partout dans le projet.

II.2. Lecture ELF bas niveau (readELF.h)

Ce module fournit une API de lecture portable des fichiers ELF, indépendante de l'endianness.

II.3. Rôle principal

— Lire correctement des entiers 8/16/32 bits
— Gérer le swap big/little endian
— Charger les différentes tables ELF

II.4. Fonctions clés

Lecture primitive :

`r_u8`, `r_u16`, `r_u32`, `r_s32`

Sections :

— `elf32_read_sht`
— `elf32_read_section`
— `elf32_section_name`

Symboles :

— `elf32_read_symtab`
— `elf32_symbol_name`

Relocations :

— `elf32_read_reltab`
— `elf32_read_relatab`
Ce module est utilisé par :
— les programmes d'affichage (phase 1)
— les modules de fusion (phase 2)

II.5. Contexte ELF et chargement objet (ELFcntx.h, elf_object.h)

II.5.a) ELFcntx

La structure `ELFcntx` encapsule :
— un lecteur ELF (`elf_reader_t`)
— l'en-tête ELF mis en cache
— un indicateur de validité
Elle permet de valider rapidement un fichier ELF32 avant traitement.

II.5.b) elf_object_t

Cette structure représente un fichier objet ELF chargé en mémoire :
— en-tête ELF
— table des sections
— table des noms de sections
— données de chaque section
Elle sert de format intermédiaire commun à toutes les étapes de fusion.

II.6. Affichage ELF (Phase 1 – readelf simplifié)

Le fichier `printELF.h` définit les fonctions d'affichage utilisées dans les étapes 1 à 5 :
— affichage de l'en-tête ELF
— affichage de la table des sections
— dump hexadécimal d'une section
— affichage de la table des symboles
— affichage des tables de relocation
Ces fonctions s'appuient sur `readELF` et `ELF.h` et produisent un affichage comparable à `arm-none-eabi-readelf`, sans chercher une stricte identité.

II.7. Fusion des sections (section_merge.h, section_merge_types.

II.7.a) Objectif

Implémenter l'étape 6 : fusion des sections PROGBITS.

II.7.b) Principe

— Les sections PROGBITS portant le même nom dans les deux fichiers sont concaténées
— Les sections présentes dans un seul fichier sont simplement copiées
— Les sections du fichier A conservent leur numérotation
— Les sections du fichier B sont remappées vers les sections de sortie

II.7.c) Structures clés

— `merged_section_t` : représente une section PROGBITS fusionnée
— `section_remap_t` :
  — mapping des indices de sections B → sortie
  — offsets de concaténation
— `merge_result_t` :
  — liste des sections de sortie
  — informations de remapping nécessaires aux étapes suivantes

II.8. Fusion des symboles (symbol_merge.h, symbol_merge_types

II.8.a) Objectif

Implémenter l'étape 7 : fusion et renumérotation des symboles.

II.8.b) Règles appliquées

- Tous les symboles locaux sont conservés
- Les symboles globaux sont fusionnés :
  - erreur si deux définitions existent
  - une seule entrée si symbole non défini
- Les indices de sections (`st_shndx`) sont mis à jour selon la fusion des sections
- Les valeurs (`st_value`) sont corrigées avec les offsets de concaténation

II.8.c) Résultat

La structure `symbol_merge_result_t` contient :
- la table des symboles fusionnée
- la table de chaînes associée
- les mappings A → sortie et B → sortie

II.9. Fusion des relocations (realocation_merge.h)

II.9.a) Objectif

Implémenter l'étape 8 : correction des relocations ARM.

II.9.b) Traitements effectués

Pour chaque entrée de relocation :
- correction de `r_offset` (concaténation des sections)
- correction de l'indice de symbole (`r_info`)
- correction de l'addend pour :
  - `R_ARM_ABS*`
  - `R_ARM_CALL`, `R_ARM_JUMP24`

Ce module dépend directement :
- du résultat de la fusion des sections
- du résultat de la fusion des symboles

II.10. Écriture du fichier ELF final (writeELF.h)

II.10.a) Objectif

Implémenter l'étape 9 : production du fichier ELF relogeable final.

II.10.b) Structure merged_elf_t

Elle contient :
- l'en-tête ELF de sortie
- la table des sections
- la table `.shstrtab`
- les sections PROGBITS fusionnées
- la table des symboles fusionnée

II.10.c) Fonctions principales

— `build_output_sections` : construit la SHT et `.shstrtab`
— `write_merged_elf` : écrit le fichier ELF final
— `merged_elf_free` : libération mémoire

III. Fonctionnement global du projet

Le fonctionnement global est le suivant :

1. Chargement des deux fichiers ELF objets
2. Fusion des sections PROGBITS
3. Fusion et correction des symboles
4. Fusion et correction des relocations
5. Construction des tables ELF de sortie
6. Écriture du fichier ELF relogeable fusionné

Note : Toutes les fonctionnalités demandées ont été complétées avec succès.

IV. Diagramme de Gantt - Progression

Le diagramme de Gantt ci-dessous illustre la répartition temporelle des différentes étapes du projet et leur avancement au fil des jours.

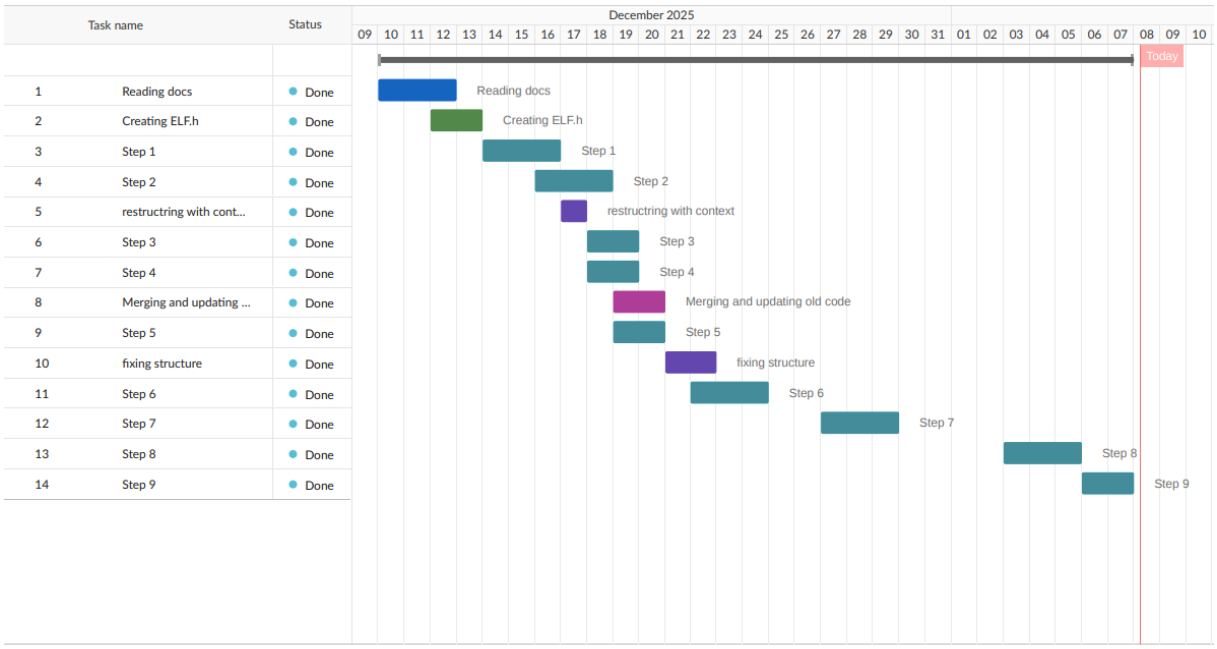

FIGURE 1 – Diagramme de Gantt du projet - Suivi temporel des étapes